NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 2: Strings, Branching, Conditionals, Indentation, and Iteration

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Ask a question ⟶ Choose a path ⟶ Repeat with purpose ⟶ Test the boundaries

About these notes

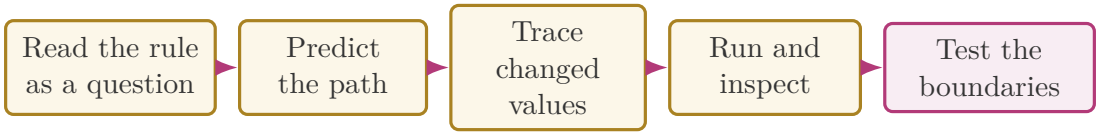
These reading notes expand the complete Week 2 lecture-slide set for *ENG 101 - Programming for Engineers*. The three lectures develop strings and Boolean questions, branching and transparent engineering decisions, and iteration with `for` and `while` loops. The notes preserve the organization, terminology, examples, engineering cautions, and classroom activities of the slides while supplying fuller explanations, annotated code, trace tables, worked answers, and additional practice.

Every Python sample in these notes contains comments. The comments explain the role of each variable, the meaning of each condition, the purpose of an indentation level, the way a loop progresses, or the reason a deliberately faulty example is incorrect. Comments support understanding, but they do not replace active practice. Students should predict the output, run the code, inspect the result, modify one feature, and explain the behavior in their own words.

Week 2 is the transition from straight-line calculation to control flow. A program can now receive text, ask a precise True-or-False question, choose among alternative actions, repeat a calculation, and stop according to a stated rule.

How to use these notes

Programming becomes understandable when you treat each example as a small experiment. The recommended Week 2 study cycle is:



For each conditional program:

1. translate the condition into a precise yes-or-no question;
2. identify which branch runs when the condition is `True` and which runs when it is `False`;
3. inspect the comparison symbols and decide whether the boundaries are inclusive or exclusive;
4. test every branch, especially values exactly at important thresholds;
5. report the calculated value and the rule, not only a label such as “PASS” or “FAIL.”

For each loop:

1. identify what repeats;
2. identify which values change and which remain fixed;
3. write the generated sequence or initial state before running;
4. trace at least the first two passes and the last pass;
5. explain why the loop must stop, or identify why it may not stop;
6. test zero passes, one pass, several passes, and the last included value where appropriate.

Engineering check

A branch implements a stated rule; it does not prove that the rule is an adequate engineering standard. A loop performs repeated steps; it does not guarantee that the step size, stopping condition, or model is physically appropriate. Human accountability remains essential.

Week 2 structure

Lecture	Main theme	Core ideas
Lecture 4	Strings and Boolean questions	Text as a Python object; text versus numbers; input and conversion; normalization; comparison operators; assignment versus equality; conditions that evaluate to `True` or `False`.
Lecture 5	Branching and engineering decision rules	`if`, `elif`, and `else`; indentation; boundary rules; logical operators; input validation; floating-point tolerances; classifications; test matrices.
Lecture 6	Iteration and repeated engineering calculations	`for`, `range()`, and `while`; loop tracing; counters and accumulators; running state; stopping conditions; `break`; `continue`; discrete design searches.

Week 2 learning outcomes

By the end of Week 2, you should be able to:

1. explain how Python represents strings and why `input()` initially returns text;
2. convert user input to an appropriate type when the intended meaning is clear;
3. create Boolean expressions using comparison and logical operators;
4. distinguish assignment `=` from equality testing `==`;
5. write, trace, and test `if`, `if-else`, and `if-elif-else` structures;
6. use indentation to define the intended branch or loop body;
7. state inclusive and exclusive boundaries explicitly;
8. validate critical inputs before carrying out risky calculations;
9. write and trace `for` loops over `range()` and over a string;
10. use counters, accumulators, and running state variables;
11. write a `while` loop with initialization, a condition, and an update;
12. identify infinite-loop risks and explain how a loop makes progress;
13. choose between `for` and `while` based on the problem structure;
14. test all important branches, boundaries, stopping conditions, and failure cases.

Key idea

The central Week 2 message is: make the rule explicit, make the boundary testable, and make the loop progress.

Contents

How to use these notes **3**
Week 2 structure . . . 4
Week 2 learning outcomes . . . 4

1 Strings and Boolean Questions **8**
1.1 The transition from calculation to decision . . . 8
1.2 Retrieval warm-up from Week 1 . . . 8
1.3 What is a string object? . . . 9
1.3.1 Text that looks numerical is still text . . . 10
1.4 Operators can have different meanings for strings . . . 10
1.5 A string has length and positions . . . 11
1.6 Why does input return a string? . . . 11
1.6.1 Convert only when the intended meaning is clear . . . 12
1.7 Normalizing simple text input . . . 13
1.8 Boolean values answer yes-or-no questions . . . 13
1.9 Comparison operators create Boolean results . . . 14
1.10 Assignment and equality are different operations . . . 14
1.11 Boolean expressions as engineering questions . . . 14
1.12 Intervals and chained comparisons . . . 15
1.13 Strings can also be compared . . . 16
1.14 Activity: predict, classify, and explain . . . 16
1.15 Bug hunt: type, quotation, comparison, and output . . . 17
1.16 Lecture 4 recap . . . 18
1.17 Lecture 4 exit ticket with solution . . . 18

2 Branching and Engineering Decision Rules **19**
2.1 Opening problem: from stress to a decision . . . 19
2.2 Straight-line execution versus branching . . . 20
2.3 The anatomy of an if statement . . . 20
2.3.1 How the first branch flows . . . 21
2.4 Indentation defines the decision block . . . 21
2.5 Activity: mark the blocks before running . . . 22
2.6 Two alternatives with if-else . . . 22
2.7 Live coding: a transparent stress pass/fail report . . . 22
2.8 Boundary values are part of the specification . . . 23
2.9 More than two outcomes with elif . . . 23
2.10 Condition order can change the answer . . . 24
2.11 Logical operators combine engineering conditions . . . 25
2.11.1 Operating limits with and, or, and not . . . 25
2.12 A classic Boolean logic bug . . . 26
2.13 Input validation protects the calculation . . . 26
2.14 Floating-point decisions usually need a tolerance . . . 27
2.15 Integrated example: factor-of-safety classifier . . . 27
2.16 Build evidence with a test matrix . . . 28
2.17 Assignment 1 launch: decisions and repetition . . . 29
2.18 Lecture 5 recap . . . 29
2.19 Lecture 5 exit ticket with solution . . . 30

3 Iteration and Repeated Engineering Calculations 31
3.1 Opening problem: one formula, many times . . . 31
3.2 Manual repetition does not scale . . . 32
3.3 Your first for loop . . . 32
3.3.1 Trace a loop one pass at a time . . . 33
3.4 Three common forms of range . . . 33
3.5 Activity: predict the sequences and pass counts . . . 34
3.6 Indentation defines the loop body . . . 34
3.7 Engineering table: velocity over time . . . 35
3.8 A decision can live inside a loop . . . 35
3.9 Counters, accumulators, and running values . . . 36
3.9.1 Accumulator example: total energy over intervals . . . 36
3.10 Strings are iterable too . . . 37
3.11 Choosing between for and while . . . 37
3.12 The anatomy of a while loop . . . 38
3.12.1 How a while loop flows . . . 39
3.13 Tank filling: a state evolves until a threshold . . . 39
3.14 Infinite loop: the condition never changes . . . 40
3.15 break and continue . . . 40
3.15.1 Using break in a small design search . . . 40
3.15.2 Using continue to skip an invalid case . . . 41
3.16 Design search: first tested area that passes . . . 41
3.17 A loop needs a defensible stopping rule . . . 42
3.18 Loop design checklist . . . 42
3.19 Laboratory 1: branching and loops . . . 43
3.20 Week 2 concept map . . . 43
3.21 Week 2 exit ticket with solution . . . 44

4 Week 2 Synthesis, Quick Reference, and Practice 45
4.1 The central Week 2 workflow . . . 45
4.2 Week 2 Python quick reference . . . 45
4.3 Decision-program checklist . . . 47
4.4 Loop-program checklist . . . 47
4.5 Glossary . . . 48
4.6 Cumulative practice . . . 49
4.6.1 Concept questions . . . 49
4.6.2 Code tracing: strings and Boolean values . . . 49
4.6.3 Code tracing: branch order . . . 50
4.6.4 Code tracing: range and accumulator . . . 50
4.6.5 Debugging practice: Boolean logic . . . 50
4.6.6 Debugging practice: loop progress . . . 51
4.6.7 Programming practice: operating-state classifier . . . 51
4.6.8 Programming practice: repeated energy calculation . . . 51
4.6.9 Programming practice: event-driven tank filling . . . 51
4.6.10 Engineering interpretation . . . 52
4.7 Practice solutions . . . 52
4.7.1 Concept responses . . . 52
4.7.2 Tracing solutions . . . 52
4.7.3 Boolean-logic repair . . . 53
4.7.4 Cooling-loop repair . . . 53
4.7.5 Operating-state classifier solution . . . 53
4.7.6 Repeated energy solution . . . 54

4.7.7 Tank-filling solution 54
4.7.8 Engineering interpretation response 55

Source Basis and Suggested Reading **56**

1 Strings and Boolean Questions

Lecture-slide alignment: Lecture 4 of the Week 2 slide set: how Python represents text, accepts input, and turns engineering questions into `True` or `False`.

Learning outcomes

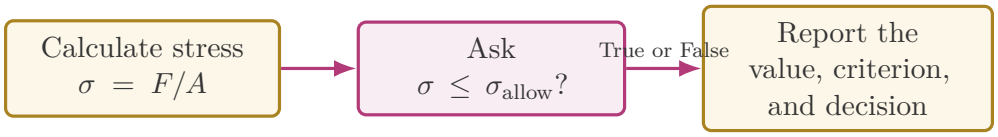
By the end of this chapter, you should be able to:

1. explain what a Python string object represents;
2. distinguish text such as `"12"` from the number `12`;
3. use simple string operations for labels, prompts, and reports;
4. explain why `input()` returns a string;
5. select `int`, `float`, or `str` according to the meaning of an input;
6. normalize simple text input using `strip()` and `lower()`;
7. create Boolean expressions using comparison operators;
8. distinguish assignment `=` from equality testing `==`;
9. read a condition as a precise engineering question;
10. interpret chained comparisons and string comparisons correctly.

1.1 The transition from calculation to decision

Week 1 focused mainly on straight-line calculations: assign values, evaluate expressions, and print results. Week 2 adds control flow. A program can now ask a question whose answer is `True` or `False`, then use that answer to choose an action.

For a stress calculation, the transition is:



Numbers store measurements and calculated quantities. Strings store labels, units, identifiers, prompts, and messages. Boolean values store the answers to precise yes-or-no questions. These types work together in a decision program.

Definition

A **Boolean expression** is an expression whose value is either `True` or `False`. It should represent a precise question such as "Is the temperature no greater than 80 °C?"

1.2 Retrieval warm-up from Week 1

Predict the output before running the following program.

```python
# Store the integer 3 using the name x.
x = 3

# Reassign x to its current value plus 2.
x = x + 2

# Display the updated numerical value.
print(x)

# Join two strings; this is text concatenation, not arithmetic.
print("3" + "2")

# Add two integer values numerically.
print(3 + 2)

# Ask Python for the type of the string literal "3".
print(type("3"))
```

Code 1.1: Retrieval warm-up with comments.

The outputs are `5`, `32`, `5`, and `<class 'str'>`. The important contrast is that `"3" + "2"` joins text, whereas `3 + 2` adds numbers. Quotation marks change the type and therefore change the operation.

1.3 What is a string object?

A Python **string** is an ordered sequence of characters. The type name is `str`. Strings are used for:

- course names, material names, and equipment labels;
- prompts and status messages;
- physical-unit labels such as `"MPa"`;
- report headings;
- file names, asset identifiers, and sample codes.

```python
# Store a course label as text.
course = "ENG 101"

# Single quotation marks also create a string.
material = 'steel'

# Store a physical-unit label; this is text, not a numerical unit object.
unit = "MPa"

# Store a message that may appear in a report.
message = "Design check complete"

# Display the course label.
print(course)
```

```
15
16 # Confirm that material is an object of type str.
17 print(type(material))
```

Code 1.2: Creating and inspecting string objects.

Quotation marks matter. The expression `steel` is interpreted as a variable name. The expression `"steel"` is a string literal. If no variable called `steel` has been assigned, using it without quotation marks produces a `NameError`.

1.3.1 Text that looks numerical is still text

Appearance does not determine type. The object type determines which operations are meaningful.

```
1  # Store the numerical integer twelve.
2  a = 12
3
4  # Store the two text characters "1" and "2".
5  b = "12"
6
7  # Display the type of each object.
8  print(type(a))
9  print(type(b))
10
11 # Multiply an integer by three; this is arithmetic.
12 print(a * 3)
13
14 # Repeat a string three times; this is text repetition.
15 print(b * 3)
```

Code 1.3: The same characters can represent different types.

```
1 <class 'int'>
2 <class 'str'>
3 36
4 121212
```

Code 1.4: Output from the numerical-text comparison.

The final line does not calculate thirty-six. It repeats the string `"12"` three times.

1.4 Operators can have different meanings for strings

An operator is **overloaded** when its meaning depends on the operand types. In the following example, `+` concatenates strings and `*` repeats a string.

```
1  # Store the first word of a report title.
2  first = "Engineering"
3
4  # Store the second word of the title.
5  second = "Computation"
```

```
6
7  # Join the two strings with one space between them.
8  course_name = first + " " + second
9
10 # Repeat a hyphen 28 times to make a report separator.
11 separator = "-" * 28
12
13 # Print a simple three-line heading.
14 print(separator)
15 print(course_name)
16 print(separator)
```

Code 1.5: String concatenation and repetition.

For numbers, `+` means addition. For strings, it means concatenation. For an integer and a string, `*` means repetition. Not every combination is meaningful.

```
1  # ERROR: Python cannot multiply one string by another string.
2  # The repetition count must be an integer, not the text "B".
3  print("A" * "B")
```

Code 1.6: A deliberately invalid string operation.

This statement raises a `TypeError`.

1.5 A string has length and positions

Because a string is an ordered sequence, its characters have positions. Python uses zero-based indexing.

```
1  # Store an equipment identifier containing eight characters.
2  code = "PUMP-07A"
3
4  # Count all characters, including the hyphen.
5  print(len(code))
6
7  # Display the first character; forward indexing starts at zero.
8  print(code[0])
9
10 # Display the final character; -1 counts from the end.
11 print(code[-1])
```

Code 1.7: String length and selected positions.

For a string of length n, the forward indices are

$$0, 1, 2, \ldots, n-1.$$

For `"PUMP-07A"`, `len(code)` is 8, `code[0]` is `"P"`, and `code[-1]` is `"A"`. Week 3 develops indexing, slicing, and string manipulation in detail; Week 2 uses only the basic idea that a string is ordered.

1.6 Why does input return a string?

The keyboard supplies characters. Python cannot infer whether the characters `12` represent:

- the number twelve;
- part of a calendar date;
- an equipment code;
- a sample label;
- a numerical quantity that still requires a unit.

Therefore, `input()` always returns an object of type `str`.

```
# Ask the user to type a voltage value.
# The prompt includes the expected physical unit.
voltage = input("Enter voltage in V: ")

# Display exactly what the user entered.
print(voltage)

# Confirm that input() returned a string.
print(type(voltage))
```

Code 1.8: Inspecting the type returned by input.

Even if the user types `12`, the stored object is the string `"12"`.

1.6.1 Convert only when the intended meaning is clear

Choose the type according to the meaning of the input:

- a measured or calculated quantity normally becomes a `float`;
- a whole-number count normally becomes an `int`;
- a name or identifier remains a `str`.

```
# Read a measured voltage and convert the returned text to float.
voltage_v = float(
    input("Enter voltage in V: ")
)

# Read a whole-number sensor count and convert the text to int.
number_of_sensors = int(
    input("Enter number of sensors: ")
)

# Keep an asset identifier as text because it is not used in arithmetic.
asset_id = input("Enter asset ID: ")
```

Code 1.9: Converting input according to engineering meaning.

A conversion may fail when the text does not represent the requested type.

```
# ERROR: the word "twelve" is not a valid floating-point literal.
voltage_v = float("twelve")
```

Code 1.10: A conversion that raises ValueError.

At this stage of the course, it is enough to recognize the error. Exception handling is developed later.

1.7 Normalizing simple text input

Users may enter the same unit in different forms: `" kPa "`, `"KPA"`, or `"kpa"`. A small normalization step makes comparison more reliable.

```
# Ask the user to enter a pressure-unit label.
unit = input("Enter pressure unit: ")

# Remove spaces around the text and convert letters to lowercase.
unit = unit.strip().lower()

# Display the normalized unit for inspection.
print(unit)
```

Code 1.11: Normalizing a unit string before comparison.

The method `strip()` removes surrounding whitespace. The method `lower()` converts letters to lowercase. Because strings are immutable, the methods return a new string, which is reassigned to `unit`.

Engineering check

Normalization should preserve the intended engineering meaning. Lowercasing a unit label may be convenient for this instructional example, but real unit symbols can be case-sensitive. For example, `Pa` and `pA` do not represent the same quantity. The programmer must decide which transformations are appropriate.

1.8 Boolean values answer yes-or-no questions

The type `bool` has exactly two values:

`True` `False`.

Useful engineering questions include:

- Is the calculated stress within the allowable limit?
- Is the sensor connected?
- Is the temperature above the alarm threshold?
- Is the entered unit recognized?

A useful condition must be unambiguous. “Is the temperature high?” is incomplete until “high” is defined by a numerical rule such as `temperature_c > 80.0`.

1.9 Comparison operators create Boolean results

Operator	Meaning	Engineering example
`<`	less than	`temperature_c < 80`
`<=`	less than or equal to	`stress_mpa <= allowable_mpa`
`>`	greater than	`speed_mps > 30`
`>=`	greater than or equal to	`factor_of_safety >= 2.0`
`==`	equal to	`mode == "auto"`
`!=`	not equal to	`material != "steel"`

Read each comparison as a question. For example,

```
stress_mpa <= allowable_mpa
```

means:

> Is the calculated stress no greater than the allowable stress?

1.10 Assignment and equality are different operations

Assignment	Equality test
`mode = "automatic"` Associates the name `mode` with a string object.	`mode == "automatic"` Asks whether the current value of `mode` equals the given string.

The following is invalid Python:

```
# ERROR: a condition needs == for comparison, not = for assignment.
if mode = "automatic":
    # This line is never reached because the if statement is invalid.
    print("Automatic mode selected")
```

Code 1.12: A common syntax error in a condition.

Use `=` to assign and `==` to compare.

1.11 Boolean expressions as engineering questions

```
# Store a calculated stress in MPa.
stress_mpa = 120.0

# Store the allowable stress in MPa.
allowable_mpa = 125.0
```

```
6
7  # Store a measured temperature in deg C.
8  temperature_c = 82.0
9
10 # Store a normalized pressure-unit label.
11 unit = "kpa"
12
13 # Ask whether the calculated stress is within the limit.
14 print(stress_mpa <= allowable_mpa)
15
16 # Ask whether the temperature exceeds 80 deg C.
17 print(temperature_c > 80.0)
18
19 # Ask whether the unit is exactly the string "kpa".
20 print(unit == "kpa")
21
22 # Ask whether the unit differs from the string "pa".
23 print(unit != "pa")
```

Code 1.13: Four Boolean questions with annotated meaning.

All four results are `True`.

For every Boolean expression, identify the left operand, operator, right operand, and resulting question. Do not treat a comparison as a mysterious symbol sequence.

1.12 Intervals and chained comparisons

Python can express a mathematical interval directly.

```
1  # Store the measured temperature in deg C.
2  temperature_c = 65.0
3
4  # Ask whether the temperature lies in the inclusive interval [20, 80].
5  within_range = 20.0 <= temperature_c <= 80.0
6
7  # Display the Boolean result.
8  print(within_range)
```

Code 1.14: An inclusive temperature interval.

This corresponds closely to

$$20 \leq T \leq 80.$$

An equivalent expanded expression is:

```
1  # Store the measured temperature in deg C.
2  temperature_c = 65.0
3
4  # Require both the lower-bound and upper-bound questions to be true.
5  within_range = (
6      temperature_c >= 20.0
7      and temperature_c <= 80.0
8  )
```

```
9
10 # Display the combined Boolean result.
11 print(within_range)
```

Code 1.15: Equivalent interval check using and.

The use of `<=` means that exactly 20 °C and exactly 80 °C are accepted. Boundary symbols are part of the specification, not decorative details.

1.13 Strings can also be compared

String comparison is case-sensitive by default. Therefore `"KPA" == "kpa"` is `False`. Normalize first when that matches the intended rule.

```
1  # Store a unit label with uppercase letters.
2  unit = "KPA"
3
4  # Remove surrounding spaces and normalize the case.
5  normalized_unit = unit.strip().lower()
6
7  # Ask whether the normalized unit is exactly "kpa".
8  is_kpa = normalized_unit == "kpa"
9
10 # Ask whether the normalized unit matches any recognized option.
11 is_recognized = (
12     normalized_unit == "pa"
13     or normalized_unit == "kpa"
14     or normalized_unit == "mpa"
15 )
16
17 # Display both Boolean results.
18 print(is_kpa)
19 print(is_recognized)
```

Code 1.16: Comparing a normalized unit with recognized labels.

Both results are `True` for the given input.

1.14 Activity: predict, classify, and explain

```
1  # Store a measured pressure in kPa.
2  pressure_kpa = 200.0
3
4  # Store the limiting pressure in kPa.
5  limit_kpa = 200.0
6
7  # Store the current control mode as text.
8  mode = "manual"
9
10 # Store one digit as text rather than as a number.
11 number_text = "5"
12
13 # Ask whether the pressure is strictly below the limit.
14 print(pressure_kpa < limit_kpa)
```

```
15
16 # Ask whether the pressure is below or equal to the limit.
17 print(pressure_kpa <= limit_kpa)
18
19 # Ask whether the mode is exactly "auto".
20 print(mode == "auto")
21
22 # Repeat the string twice.
23 print(number_text * 2)
24
25 # Convert the string to int, then multiply numerically.
26 print(int(number_text) * 2)
```

Code 1.17: Activity code with comments.

The outputs are `False`, `True`, `False`, `55`, and `10`. The first two lines show why equality at a boundary matters. The final two lines show how explicit conversion changes string repetition into numerical multiplication.

1.15 Bug hunt: type, quotation, comparison, and output

The following program contains four different problems.

```
1  # BUG 1: input() returns str, but this value is later compared with an int.
2  voltage = input("Enter voltage in V: ")
3
4  # BUG 2: kPa lacks quotation marks and is treated as an undefined name.
5  unit = kPa
6
7  # BUG 3: comparing a string voltage with the integer 12 raises TypeError.
8  is_high = voltage > 12
9
10 # BUG 4: this uses assignment instead of equality testing.
11 is_kpa = unit = "kPa"
12
13 # BUG 5 in the slide activity: quotation marks print literal text, not the variable
       value.
14 print("is_high")
15
16 # This line would display the value if execution reached it.
17 print(is_kpa)
```

Code 1.18: Deliberately faulty input and comparison program.

The slide asks students to identify four categories; the code actually exposes a closely connected fifth issue in the printed output. A repaired version is:

```
1  # Convert the entered voltage text to a floating-point number.
2  voltage_v = float(input("Enter voltage in V: "))
3
4  # Store the unit label as a string.
5  unit = "kPa"
6
```

```
# Compare two numerical values of compatible type.
is_high = voltage_v > 12.0

# Compare the unit string with the expected text using ==.
is_kpa = unit == "kPa"

# Print the Boolean variables, not their names as literal text.
print(is_high)
print(is_kpa)
```

Code 1.19: Corrected and annotated version of the bug-hunt program.

1.16 Lecture 4 recap

Lecture 4 connects four ideas:

- strings represent prompts, labels, identifiers, units, and messages;
- user input begins as text and is converted only when the intended type is clear;
- comparison operators transform numerical or text relationships into Boolean values;
- the chosen comparison symbols determine the exact boundary rule.

Section summary

Week 1 taught Python to calculate. Lecture 4 begins teaching Python to formulate precise questions. Those questions become the conditions used by branches and loops in the rest of Week 2.

1.17 Lecture 4 exit ticket with solution

For the inclusive interval from 20 °C to 80 °C, a clear Boolean expression is:

```
# Store a temperature in deg C for testing.
temperature_c = 80.0

# Accept both boundaries by using <= on each side.
is_acceptable = 20.0 <= temperature_c <= 80.0

# At exactly 80 deg C, the result is True.
print(is_acceptable)
```

Code 1.20: Inclusive temperature-range exit-ticket solution.

At 80 °C, the result is `True`. A single `=` would be wrong because it performs assignment rather than equality testing. In a condition, use comparisons such as `==`, `<=`, or `>=`.

2 Branching and Engineering Decision Rules

Lecture-slide alignment: Lecture 5 of the Week 2 slide set: using `if`, `elif`, `else`, indentation, and logical operators to encode transparent engineering decisions.

Learning outcomes

By the end of this chapter, you should be able to:

1. explain how branching changes the flow of a program;
2. write and trace `if`, `if-else`, and `if-elif-else` structures;
3. use indentation to define the intended code block;
4. combine conditions with `and`, `or`, and `not`;
5. order classification rules correctly;
6. validate simple engineering inputs before calculation;
7. compare floating-point values using a problem-appropriate tolerance;
8. construct representative, boundary, extreme, and invalid-input tests;
9. distinguish a program decision from proof of real-world safety.

2.1 Opening problem: from stress to a decision

A steel bracket has:

- yield strength $\sigma_y = 250\,\text{MPa}$;
- required factor of safety $\text{FoS}_{\text{req}} = 2.0$;
- calculated stress $\sigma = 120\,\text{MPa}$.

The allowable stress under this simplified rule is

$$\sigma_{\text{allow}} = \frac{\sigma_y}{\text{FoS}_{\text{req}}} = \frac{250\,\text{MPa}}{2.0} = 125\,\text{MPa}.$$

Because $120\,\text{MPa} \leq 125\,\text{MPa}$, the bracket passes this stated numerical rule. A useful program should still ask four questions:

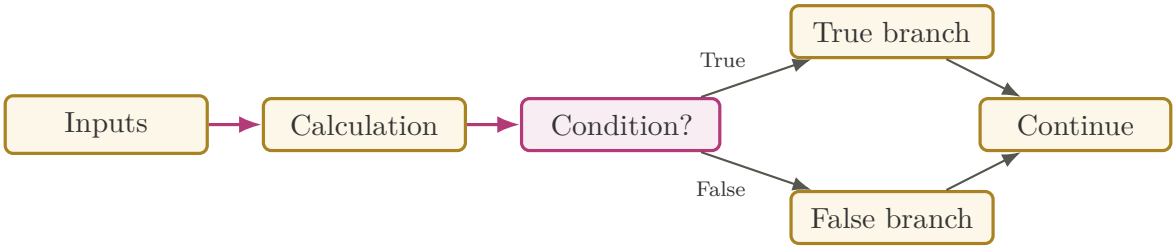
1. Are the inputs physically meaningful?
2. Does the calculated stress satisfy the chosen criterion?
3. Which numerical values and assumptions should be reported?
4. What limitations remain outside the simplified model?

Engineering check

A pass according to one simplified stress rule is not proof that a real bracket is safe. Bending, fatigue, stress concentration, material variability, manufacturing defects, loading uncertainty, and the suitability of the selected factor of safety may also matter.

2.2 Straight-line execution versus branching

A straight-line program executes one statement after another in a fixed order. A branching program evaluates a condition and follows one of several possible paths.



Branching does not mean that all paths execute. In an `if-else` structure, exactly one of the two alternative blocks runs.

2.3 The anatomy of an if statement

```
# Store the calculated factor of safety.
factor_of_safety = 2.3

# Ask whether the value meets or exceeds the stated minimum.
if factor_of_safety >= 2.0:
    # This line runs only when the condition is True.
    print("Design passes the stated rule.")

# This unindented line is outside the if block and always runs.
print("Calculation complete.")
```

Code 2.1: A first if statement with comments.

An `if` statement has three essential syntax elements:

1. the keyword `if`;
2. a Boolean condition followed by a colon;
3. an indented block of one or more statements.

Read the statement aloud:

> If the factor of safety is at least 2.0, print the pass message. Then, whether or not the condition was true, print that the calculation is complete.

2.3.1 How the first branch flows

When Python reaches the `if`:

1. evaluate the condition;
2. if it is `True`, execute the indented block;
3. if it is `False`, skip the block;
4. continue with the first unindented statement after the block.

2.4 Indentation defines the decision block

Indentation is part of Python's meaning. Compare the two programs.

```python
# Store the measured temperature in deg C.
temperature_c = 95.0

# Ask whether the temperature exceeds the stated limit.
if temperature_c > 80.0:
    # Both indented messages belong to the warning branch.
    print("WARNING")
    print("Improve cooling")

# This completion message is outside the branch and always appears.
print("Check complete")
```

Code 2.2: Correct indentation: both warnings belong to the condition.

```python
# Store a temperature below the warning threshold.
temperature_c = 60.0

# The condition is False, so only the first indented line is skipped.
if temperature_c > 80.0:
    # This warning correctly belongs to the conditional block.
    print("WARNING")

# LOGIC ERROR: this line is unindented and therefore always runs.
print("Improve cooling")

# This line also always runs.
print("Check complete")
```

Code 2.3: Misleading indentation: one warning is outside the condition.

The second program runs without a syntax error, but the output is misleading. A logic error can therefore arise from indentation even when Python accepts the program.

Do not infer a block from visual proximity alone. Count indentation levels. Statements aligned at the same indentation level belong to the same structural level.

2.5 Activity: mark the blocks before running

```
# Store the calculated stress and allowable stress in MPa.
stress_mpa = 140.0
allowable_mpa = 125.0

# Ask whether the stress passes the stated limit.
if stress_mpa <= allowable_mpa:
    # These two lines run only if the condition is True.
    print("Stress check passed")
    print("Within allowable limit")

# This final line is outside the conditional block.
print("End of report")
```

Code 2.4: Conditional-block tracing activity.

The condition is `False`, so the two indented messages are skipped. The output contains only `End of report`. If the second message were accidentally unindented, the report could claim `Within allowable limit` even though the stress exceeded the limit.

2.6 Two alternatives with if-else

Use `if-else` when exactly one of two alternatives must run.

```
# Store the calculated factor of safety.
factor_of_safety = 1.6

# Ask whether the factor of safety meets the minimum value.
if factor_of_safety >= 2.0:
    # This branch runs when the condition is True.
    print("Design is acceptable")
else:
    # This branch handles every remaining case.
    print("Design is not acceptable")
```

Code 2.5: A two-way factor-of-safety decision.

No condition follows `else`; it means "all remaining cases." After one branch runs, execution continues below the complete structure.

2.7 Live coding: a transparent stress pass/fail report

A strong engineering report displays both the calculated value and the criterion used for the decision.

```
# Store the material yield strength in MPa.
yield_strength_mpa = 250.0

# Store the required factor of safety.
required_fos = 2.0

# Store the calculated operating stress in MPa.
```

```
stress_mpa = 120.0

# Calculate the allowable stress under the stated simplified rule.
allowable_mpa = yield_strength_mpa / required_fos

# Print the evidence used by the branch.
print("STRESS CHECK")
print(f"Stress: {stress_mpa:.2f} MPa")
print(f"Allowable stress: {allowable_mpa:.2f} MPa")

# Compare compatible quantities expressed in the same unit.
if stress_mpa <= allowable_mpa:
    # Equality is accepted because the rule uses <=.
    print("Decision: PASS")
else:
    # This branch handles stresses above the allowable value.
    print("Decision: FAIL")
```

Code 2.6: Transparent stress pass/fail report.

The report is stronger than printing only `PASS`. Another engineer can inspect both numbers and see why the branch was selected.

2.8 Boundary values are part of the specification

Compare two possible rules:

Strict rule	Inclusive rule
`stress_mpa < allowable_mpa`	`stress_mpa <= allowable_mpa`
Equality fails.	Equality passes.
Only a strictly lower stress passes.	A stress equal to the limit passes.

At $\sigma = \sigma_{\text{allow}}$, the two rules produce different results. The code must match the intended engineering specification.

Decision rule

Before coding a boundary, write the rule in words. Decide whether "at the limit" belongs to the acceptable or unacceptable category. Then choose `<`, `<=`, `>`, or `>=` accordingly.

2.9 More than two outcomes with elif

Use `elif` when a value can fall into several mutually exclusive categories.

```
# Store the calculated factor of safety.
factor_of_safety = 2.8

# Test thresholds from the lowest category upward.
if factor_of_safety < 1.0:
    # Values below 1.0 enter the first category.
    category = "failure expected"
elif factor_of_safety < 2.0:
```

```
    # Reaching this branch means FoS is at least 1.0 but below 2.0.
    category = "insufficient margin"
elif factor_of_safety < 4.0:
    # Reaching this branch means FoS is at least 2.0 but below 4.0.
    category = "acceptable by stated rule"
else:
    # All remaining values are 4.0 or greater.
    category = "conservative"

# Print the selected category after the branch chain finishes.
print(category)
```

Code 2.7: Four-category factor-of-safety classifier.

Python tests conditions from top to bottom and selects the first one that is `True`. Later branches are skipped.

For `factor_of_safety = 2.8`, the first two conditions are false and the third is true, so the category is `"acceptable by stated rule"`.

2.10 Condition order can change the answer

The following order is incorrect for increasing thresholds.

```
# Store a low factor of safety.
fos = 1.2

# LOGIC ERROR: this broad condition is already True for 1.2.
if fos < 4.0:
    category = "acceptable"
elif fos < 2.0:
    # This branch is never reached for values below 2.0,
    # because they also satisfy fos < 4.0 first.
    category = "unsafe"
else:
    category = "conservative"

# The misleading result is "acceptable".
print(category)
```

Code 2.8: Incorrect branch order.

A better order is:

```
# Store the same factor of safety.
fos = 1.2

# Test the most restrictive lower category first.
if fos < 1.0:
    category = "failure expected"
elif fos < 2.0:
    category = "unsafe"
elif fos < 4.0:
    category = "acceptable"
else:
    category = "conservative"
```

```
13
14 # The correct category for 1.2 is "unsafe".
15 print(category)
```

Code 2.9: Correct branch order from restrictive to broad.

Key idea

For increasing numerical thresholds, test the lowest and most restrictive category first. Each later branch then represents the values not already captured by earlier conditions.

2.11 Logical operators combine engineering conditions

Operator	Meaning	Example rule
`and`	True only when both conditions are true.	Temperature and pressure must both remain within limits.
`or`	True when at least one condition is true.	Warn if temperature or pressure exceeds a limit.
`not`	Reverses a Boolean value.	Report an error if the sensor is not connected.

Translate the decision rule into plain language before combining expressions.

2.11.1 Operating limits with and, or, and not

```
1  # Store measured operating values.
2  temperature_c = 75.0
3  pressure_kpa = 180.0
4
5  # Store a Boolean representing sensor availability.
6  sensor_connected = True
7
8  # Give important subconditions meaningful names.
9  temperature_ok = temperature_c <= 80.0
10 pressure_ok = pressure_kpa <= 200.0
11
12 # Require all three conditions to be True.
13 if temperature_ok and pressure_ok and sensor_connected:
14     print("System is within stated limits")
15
16 # Reverse the sensor-connected value to detect disconnection.
17 if not sensor_connected:
18     print("ERROR: sensor disconnected")
19
20 # Warn when either numerical limit is exceeded.
21 if temperature_c > 80.0 or pressure_kpa > 200.0:
22     print("WARNING: at least one limit exceeded")
```

Code 2.10: Combining several operating conditions.

Meaningful Boolean names such as `temperature_ok` and `pressure_ok` make a rule easier to read, test, and explain.

2.12 A classic Boolean logic bug

A common mistake is to write:

```
# Store an unrecognized pressure unit.
unit = "psi"

# LOGIC ERROR: only the first part compares unit with a string.
# The nonempty strings "kpa" and "mpa" are truthy by themselves.
if unit == "pa" or "kpa" or "mpa":
    print("Recognized unit")
else:
    print("Unknown unit")
```

Code 2.11: Incorrect unit-recognition condition.

This condition is effectively always true because nonempty strings have a truthy value. The clear correction repeats the comparison:

```
# Store the unit to classify.
unit = "psi"

# Compare unit explicitly with every accepted option.
if (
    unit == "pa"
    or unit == "kpa"
    or unit == "mpa"
):
    print("Recognized unit")
else:
    print("Unknown unit")
```

Code 2.12: Correct unit-recognition condition.

For `"psi"`, the corrected program prints `Unknown unit`.

2.13 Input validation protects the calculation

Some values are invalid before the main formula is evaluated. Examples include nonpositive area, zero resistance, negative mass in a classical mass input, or a temperature outside the model's stated domain.

```
# Read the applied force in newtons.
force_n = float(input("Enter force in N: "))

# Read the cross-sectional area in square millimetres.
area_mm2 = float(input("Enter area in mm^2: "))

# Reject an area that would be physically invalid or cause division by zero.
if area_mm2 <= 0:
    print("ERROR: area must be positive")
else:
    # Convert square millimetres to square metres.
    area_m2 = area_mm2 / 1_000_000

    # Calculate stress in Pa, then convert the result to MPa.
```

```
    stress_mpa = force_n / area_m2 / 1_000_000

    # Report the calculated stress only for valid area input.
    print(f"Stress = {stress_mpa:.2f} MPa")
```

Code 2.13: Validating area before calculating stress.

Validation should occur before risky operations. It prevents meaningless arithmetic and gives the user a clearer explanation than an unhandled runtime error.

Engineering check

Validation rules must themselves be justified. "Area must be positive" is generally defensible for this model. A rule such as "temperature must be below 100" would require a stated unit, model domain, and engineering reason.

2.14 Floating-point decisions usually need a tolerance

Floating-point values are finite approximations. Therefore exact equality can fail even when two values are mathematically intended to match.

```
# Add two floating-point approximations.
x = 0.1 + 0.2

# Exact equality is False for this stored result.
print(x == 0.3)

# Ask whether the absolute difference is smaller than a chosen tolerance.
print(abs(x - 0.3) < 1e-12)
```

Code 2.14: Exact equality versus tolerance.

The outputs are `False` and `True`. The tolerance `1e-12` is illustrative, not universally appropriate. A defensible tolerance depends on the scale of the problem, measurement precision, model accuracy, and required decision reliability.

Decision rule

For many numerical decisions, ask "Is the difference acceptably small?" rather than "Are the stored binary values identical?"

2.15 Integrated example: factor-of-safety classifier

```
# Store the applied axial force in newtons.
force_n = 10_000.0

# Store the cross-sectional area in square millimetres.
area_mm2 = 80.0

# Store the material yield strength in MPa.
yield_strength_mpa = 250.0

# Convert the area from mm^2 to m^2.
```

```python
area_m2 = area_mm2 / 1_000_000

# Calculate stress in Pa and convert it to MPa.
stress_mpa = force_n / area_m2 / 1_000_000

# Calculate factor of safety using compatible MPa units.
fos = yield_strength_mpa / stress_mpa

# Classify the result using ordered thresholds.
if fos < 1.0:
    category = "failure expected"
elif fos < 2.0:
    category = "insufficient margin"
elif fos < 4.0:
    category = "acceptable by stated rule"
else:
    category = "conservative"

# Print the numerical evidence and selected category.
print(f"Stress = {stress_mpa:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
print(f"Classification = {category}")
```

Code 2.15: Complete factor-of-safety calculation and classification.

For the given inputs:

$$A = 80 \text{ mm}^2 = 80 \times 10^{-6} \text{ m}^2,$$

$$\sigma = \frac{10{,}000}{80 \times 10^{-6}} = 125 \text{ MPa},$$

and

$$\text{FoS} = \frac{250}{125} = 2.00.$$

Because the first two tests use strict `<` comparisons, a value exactly equal to 2.0 reaches the third branch: `"acceptable by stated rule"`.

Engineering check

The labels in this example are an instructional classification, not a universal design standard. The result is valid only relative to the stated model, inputs, and thresholds.

2.16 Build evidence with a test matrix

A single successful run is weak evidence. Test every branch, important boundary, and invalid input that the program is expected to handle.

Test type	Input	Expected result	Why it matters
Representative	FoS = 2.8	acceptable	Normal intended use.
Boundary	FoS = 2.0	acceptable	Checks inclusion at the threshold.
Just below	FoS = 1.999	insufficient	Checks operator choice and branch order.
Invalid	$A = 0$	error before calculation	Prevents meaningless division.
Extreme	FoS = 10	conservative	Checks the final `else` branch.

A test matrix makes the intended behavior explicit before or alongside execution.

2.17 Assignment 1 launch: decisions and repetition

The slide set presents the following general task pattern for Assignment 1:

1. define inputs and units, then validate critical values;
2. calculate one or more engineering quantities;
3. classify the result using transparent branches;
4. repeat a related calculation and print a clear report.

Evidence should include:

- a plain-language algorithm and readable code;
- representative, boundary, and invalid-input tests;
- a brief engineering interpretation;
- disclosure of permitted assistance according to the assignment rules.

The authoritative Assignment 1 scope, deadline, submission format, and permitted-resource rules are those posted on Moodle. These reading notes explain the programming ideas but do not replace the final assessment instructions.

2.18 Lecture 5 recap

A reliable decision program:

- states the rule and boundary clearly;
- uses correct comparison and Boolean logic;
- indents every block intentionally;
- orders overlapping thresholds correctly;

- validates inputs before risky operations;
- reports numerical evidence, not only a classification label;
- is tested across every branch and important boundary;
- states the limitations of the model and decision rule.

2.19 Lecture 5 exit ticket with solution

The requested pressure categories are:

- below 100 kPa: low;
- 100 kPa to 200 kPa inclusive: normal;
- above 200 kPa: high.

A clear implementation is:

```
# Store a pressure in kPa for classification.
pressure_kpa = 150.0

# Test the lower category first.
if pressure_kpa < 100.0:
    category = "low"
elif pressure_kpa <= 200.0:
    # Reaching this branch means pressure is at least 100 kPa.
    # The <= operator includes exactly 200 kPa in the normal range.
    category = "normal"
else:
    # All remaining values are above 200 kPa.
    category = "high"

# Display the category selected by the ordered rules.
print(category)
```

Code 2.16: Three-category pressure classifier.

The two boundary cases are 100 kPa and 200 kPa. Both belong to the normal category under the stated inclusive rule.

3 Iteration and Repeated Engineering Calculations

Lecture-slide alignment: Lecture 6 of the Week 2 slide set: using `for` and `while` loops to express time evolution, repeated measurements, design searches, and stopping conditions.

Learning outcomes

By the end of this chapter, you should be able to:

1. explain why engineering programs need repetition;
2. write and trace a `for` loop over `range()`;
3. explain why the stop value of `range()` is excluded;
4. distinguish code inside and outside a loop body;
5. use counters, accumulators, and running values;
6. iterate over the characters of a string;
7. write a `while` loop using initialization, condition, and update;
8. identify an infinite loop and explain how a loop makes progress;
9. choose between `for` and `while`;
10. use `break` and `continue` carefully;
11. test loop boundaries, failure cases, step size, and stopping conditions.

3.1 Opening problem: one formula, many times

A ball is dropped from an initial height $h_0 = 100\,\text{m}$. Ignoring air resistance, its height is modeled by

$$h(t) = h_0 - \frac{1}{2}gt^2, \qquad g = 9.81\,\text{m/s}^2.$$

The task is to print the height at each integer second and stop after the ball reaches the ground. One calculation is not enough because the same model must be evaluated at many values of t. We also do not initially know the exact integer sample at which the calculated height becomes nonpositive.

A plain-language algorithm is:

1. store h_0 and g;
2. begin with $t = 0$;
3. calculate $h(t)$;
4. report the time and height;
5. increase time;

6. repeat until the ground condition is reached.

This recurring pattern appears throughout engineering:

- **Across time:** cooling, charging, tank filling, and motion;
- **Across cases:** load levels, trial dimensions, operating points, and cost alternatives;
- **Across data:** characters, sensor readings, samples, and records.

Key idea

A loop repeats a defined block, changes one or more values, and stops according to a clear rule.

3.2 Manual repetition does not scale

A program can calculate several heights using copied statements:

```
# Store the initial height in metres.
h0_m = 100.0

# Store gravitational acceleration in m/s^2.
g_mps2 = 9.81

# Evaluate the same model separately at four times.
height_0_s_m = h0_m - 0.5 * g_mps2 * 0**2
height_1_s_m = h0_m - 0.5 * g_mps2 * 1**2
height_2_s_m = h0_m - 0.5 * g_mps2 * 2**2
height_3_s_m = h0_m - 0.5 * g_mps2 * 3**2
```

Code 3.1: Manual repetition of the dropped-object formula.

This works for a few cases but becomes repetitive, difficult to modify, and vulnerable to copy-and-paste errors. A loop expresses the repeated structure directly.

```
# Store the fixed model parameters.
h0_m = 100.0
g_mps2 = 9.81

# Generate the integer times 0, 1, 2, and 3 seconds.
for time_s in range(0, 4):
    # Evaluate the same height model at the current time.
    height_m = h0_m - 0.5 * g_mps2 * time_s**2

    # Display the current time and calculated height.
    print(time_s, height_m)
```

Code 3.2: Replacing repeated lines with a for loop.

The loop is shorter, easier to extend, easier to inspect, and less likely to contain inconsistencies.

3.3 Your first for loop

```python
# Generate the integers 0 through 4.
for time_s in range(0, 5):
    # Print the current loop-variable value once per pass.
    print(time_s)

# This line is outside the loop and prints only once.
print("Loop finished")
```

Code 3.3: A minimal for loop over range.

Read it aloud:

> For each value of `time_s` from 0 up to, but not including, 5, print the value. After the sequence is exhausted, print `Loop finished` once.

The output values are

$$0, 1, 2, 3, 4.$$

3.3.1 Trace a loop one pass at a time

A trace table records the loop variable and action on every pass.

Pass	Value of `time_s`	Action
1	0	print 0
2	1	print 1
3	2	print 2
4	3	print 3
5	4	print 4

Hand simulation means pretending to be the interpreter. It is one of the most effective ways to understand loops, especially when several values change on each pass.

3.4 Three common forms of range

Form	Meaning	Generated values
`range(5)`	Start at 0; increase by 1; stop before 5.	0, 1, 2, 3, 4
`range(2, 7)`	Start at 2; increase by 1; stop before 7.	2, 3, 4, 5, 6
`range(0, 11, 2)`	Start at 0; increase by 2; stop before 11.	0, 2, 4, 6, 8, 10
`range(5, 0, -1)`	Start at 5; decrease by 1; stop above 0.	5, 4, 3, 2, 1

The stop value is excluded. To include $t = 10$ in an integer table beginning at 0, use `range(0, 11)`.

The full pattern is:

```
range(start, stop, step)
```

The step must move in a direction consistent with the start and stop values. For example, a negative step is needed to count downward.

3.5 Activity: predict the sequences and pass counts

```python
# Loop A: use the default start 0 and step 1.
for i in range(4):
    print(i)

# Loop B: start at 1 and stop before 6.
for j in range(1, 6):
    print(j)

# Loop C: start at 10 and subtract 3 each pass.
for k in range(10, 1, -3):
    print(k)
```

Code 3.4: Three range-prediction exercises.

Loop A generates 0, 1, 2, 3 and has four passes; the excluded stop is 4. Loop B generates 1, 2, 3, 4, 5 and has five passes; the excluded stop is 6. Loop C generates 10, 7, 4 and has three passes; the next value would be 1, which is not greater than the stop value 1, so it is excluded.

3.6 Indentation defines the loop body

Compare the following structures.

```python
# Repeat the complete indented block three times.
for time_s in range(3):
    # This message appears once per pass.
    print("Inside loop")

    # The current time value also appears once per pass.
    print(time_s)

# This line is outside the loop and appears once.
print("After loop")
```

Code 3.5: Two statements repeated together.

```python
# Repeat only the indented print statement three times.
for time_s in range(3):
    print("Inside loop")

# This line is outside the loop and prints the final stored value, 2.
print(time_s)

```

```
# This line also runs once after the loop.
print("After loop")
```

Code 3.6: Only one statement belongs to the loop.

Indentation controls which statements repeat. Incorrect indentation may produce a program that runs but does not perform the intended repeated calculation.

3.7 Engineering table: velocity over time

For constant acceleration from rest,

$$v(t) = at.$$

The following program evaluates the model at integer times from 0 to 10 seconds.

```
# Store constant acceleration in m/s^2.
acceleration_mps2 = 2.0

# Print a readable table heading.
print("Time (s)   Velocity (m/s)")
print("------------------------")

# Include integer times from 0 through 10 seconds.
for time_s in range(0, 11):
    # Evaluate v = a*t at the current loop-variable value.
    velocity_mps = acceleration_mps2 * time_s

    # Print aligned time and velocity columns.
    print(f"{time_s:8d}   {velocity_mps:14.2f}")
```

Code 3.7: Velocity table for constant acceleration.

On each pass, `time_s` acts as a new model input. The fixed parameter `acceleration_mps2` does not change. A trend check predicts that velocity should increase linearly because acceleration is constant and positive.

3.8 A decision can live inside a loop

A branch inside a loop is evaluated once on every pass.

```
# Store the dropped object's initial height in metres.
h0_m = 100.0

# Store gravitational acceleration in m/s^2.
g_mps2 = 9.81

# Evaluate the model at integer times from 0 through 5 seconds.
for time_s in range(0, 6):
    # Calculate the height at the current time.
    height_m = h0_m - 0.5 * g_mps2 * time_s**2

    # Classify the sampled state.
    if height_m > 0:
        status = "above ground"
```

```
    else:
        status = "ground reached"

    # Report the time, calculated height, and classification.
    print(
        f"t={time_s:2d} s, "
        f"h={height_m:7.2f} m, "
        f"{status}"
    )
```

Code 3.8: Classifying dropped-object height at each sampled time.

The discrete table may move from a positive height at one integer second to a negative height at the next. The actual impact occurs between the samples. The program classifies sampled model values; it does not determine the exact impact time.

Engineering check

A discrete step can jump over an event. Smaller time steps provide a finer estimate but require more calculations. This issue becomes central in later numerical-method and simulation work.

3.9 Counters, accumulators, and running values

Three common loop patterns are:

Counter
Counts events or cases. Typical update: `count += 1`.

Accumulator
Builds a running total. Typical update: `total += value`.

Running state
Updates a system quantity. Typical update: `volume += rate * dt`.

The update operators are abbreviations:

`count += 1` means `count = count + 1`.

Initialization normally occurs before the loop. A counter commonly starts at 0, a total commonly starts at 0.0, and a running state begins from the physical initial condition.

3.9.1 Accumulator example: total energy over intervals

A device operates at constant power $P = 1.5\,\text{kW}$ for five one-hour intervals. The energy added each interval is

$$\Delta E = P\Delta t.$$

```
# Store constant power in kW.
power_kw = 1.5

# Store the duration of each interval in hours.
interval_h = 1.0

```

```
# Initialize total energy before the loop begins.
total_energy_kwh = 0.0

# Process intervals 1 through 5.
for hour in range(1, 6):
    # Calculate energy added during the current interval.
    energy_this_interval = power_kw * interval_h

    # Add the new interval energy to the running total.
    total_energy_kwh += energy_this_interval

    # Report the interval number and cumulative energy.
    print(hour, total_energy_kwh)
```

Code 3.9: Accumulating total energy over five intervals.

The accumulator values are 1.5, 3.0, 4.5, 6.0, and 7.5 kWh.

If `total_energy_kwh = 0.0` were placed inside the loop, the total would reset to zero on every pass. The program would report only the energy from the current interval rather than the cumulative energy.

3.10 Strings are iterable too

A `for` loop can process a string one character at a time.

```
# Store a sample identifier as an ordered string.
sample_code = "SAMPLE-2026-A7"

# Initialize the number of detected digits.
digit_count = 0

# Bind character successively to every character in the string.
for character in sample_code:
    # Ask whether the current character is a decimal digit.
    if character.isdigit():
        # Increase the counter only for digit characters.
        digit_count += 1

# Report the final number of digits.
print(f"Number of digits = {digit_count}")
```

Code 3.10: Counting digits in a sample identifier.

The string contains the digits 2, 0, 2, 6, and 7, so the final count is 5. This integrates three Week 2 ideas: a string supplies values, a loop repeats, and a conditional decides whether to update a counter.

3.11 Choosing between for and while

Use `for` when:

- a sequence is known;
- the number of cases is predetermined;
- the program iterates over a string or collection;
- a table uses a defined set of input values.

Use `while` when:

- repetition continues until an event occurs;
- the required number of passes is not known in advance;
- an error must fall below a tolerance;
- a system state must reach a threshold.

Both loop types can often express the same task. Choose the form that communicates the problem most clearly.

3.12 The anatomy of a while loop

A `while` loop normally contains three connected parts: initialization, condition, and update.

```
# Initialization: begin at zero seconds.
time_s = 0

# Condition: repeat while time is no greater than 10 seconds.
while time_s <= 10:
    # Use or report the current state.
    print(time_s)

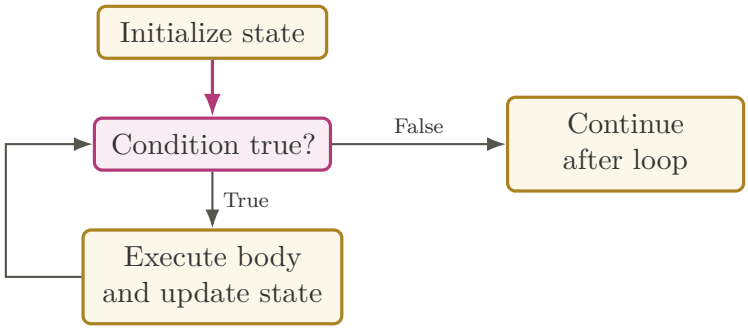
    # Update: move the control variable toward termination.
    time_s = time_s + 1
```

Code 3.11: Initialization, test, and update in a while loop.

Ask three questions:

1. What is the initial state?
2. What condition keeps the loop running?
3. What changes so that the condition eventually becomes false?

3.12.1 How a while loop flows



The condition is checked before every pass. Therefore, a `while` loop may execute zero times if its condition is false at the beginning.

3.13 Tank filling: a state evolves until a threshold

A tank has capacity 100 L and fills at 8 L/min. The program advances in steps of $\Delta t = 0.1$ min.

```
# Store the tank capacity in litres.
capacity_l = 100.0

# Store the constant inflow rate in litres per minute.
flow_rate_lpm = 8.0

# Initialize the tank as empty.
volume_l = 0.0

# Initialize elapsed time in minutes.
time_min = 0.0

# Choose the simulation time step in minutes.
dt_min = 0.1

# Repeat until the calculated volume reaches or exceeds capacity.
while volume_l < capacity_l:
    # Advance the simulation clock by one time step.
    time_min += dt_min

    # Add inflow rate times time step to the running volume.
    volume_l += flow_rate_lpm * dt_min

# Report the first sampled time at which capacity was reached.
print(f"Capacity reached at about {time_min:.1f} min")

# Report the final calculated volume, which may slightly exceed capacity.
print(f"Calculated volume = {volume_l:.2f} L")
```

Code 3.12: Tank-filling simulation with a threshold condition.

The progress variable is `volume_l`. It increases on every pass and therefore moves toward the stopping threshold.

The exact analytical fill time is

$$t = \frac{100}{8} = 12.5\,\text{min}.$$

With a time step of 0.1 min, the sampled result should be close to this value. A smaller step gives a finer estimate but requires more passes.

3.14 Infinite loop: the condition never changes

```
# Initialize the loop-control variable.
time_s = 0

# The condition is initially True.
while time_s <= 10:
    # This line repeatedly prints zero.
    print(time_s)

    # BUG: there is no update, so time_s remains zero forever.
```

Code 3.13: A deliberately infinite loop.

Because `time_s` never changes, the condition remains true. A useful debugging checklist is:

- Which variable controls the condition?
- Where is it initialized?
- Where is it updated?
- Does the update move toward termination?

A repaired version is:

```
# Initialize the loop-control variable.
time_s = 0

# Repeat while the inclusive upper boundary is satisfied.
while time_s <= 10:
    # Report the current time.
    print(time_s)

    # Increase time so the condition eventually becomes False.
    time_s += 1
```

Code 3.14: Corrected loop with progress toward termination.

3.15 break and continue

The statement `break` exits the nearest loop immediately. The statement `continue` skips the remaining statements in the current pass and proceeds to the next pass.

3.15.1 Using break in a small design search

```python
# Test areas from 100 to 1000 mm^2 in increments of 50 mm^2.
for area_mm2 in range(100, 1001, 50):
    # In N/mm^2, the numerical result is directly MPa.
    stress_mpa = 8000 / area_mm2

    # Ask whether the current candidate meets the stress limit.
    if stress_mpa <= 50:
        # Report the first passing candidate.
        print(area_mm2)

        # Exit the loop because the search goal has been met.
        break
```

Code 3.15: Stop at the first tested area that meets a stress rule.

3.15.2 Using continue to skip an invalid case

```python
# Generate areas including an invalid zero value.
for area_mm2 in range(0, 501, 100):
    # Skip the rest of this pass when division would be invalid.
    if area_mm2 == 0:
        continue

    # Calculate stress only for positive tested areas.
    stress_mpa = 5000 / area_mm2

    # Report the valid candidate and calculated stress.
    print(area_mm2, stress_mpa)
```

Code 3.16: Skip a zero-area candidate before division.

These statements can make a small search clearer, but excessive jumps can make control flow difficult to reason about.

3.16 Design search: first tested area that passes

The following program searches a discrete set of candidate areas from smallest to largest.

```python
# Store the applied force in newtons.
force_n = 8000.0

# Store the allowable stress in pascals.
allowable_stress_pa = 50e6

# Use None to mean that no acceptable area has been found yet.
required_area_mm2 = None

# Test candidate areas from 100 to 1000 mm^2 in steps of 50 mm^2.
for area_mm2 in range(100, 1001, 50):
    # Convert the current candidate area to m^2.
    area_m2 = area_mm2 / 1_000_000

    # Calculate the axial stress in pascals.
    stress_pa = force_n / area_m2
```

```
17
18      # Stop at the first candidate that meets the allowable-stress rule.
19      if stress_pa <= allowable_stress_pa:
20          required_area_mm2 = area_mm2
21          break
22
23  # Check whether the search ended without finding a passing candidate.
24  if required_area_mm2 is None:
25      print("No tested area passed")
26  else:
27      # Report the smallest passing area in the tested discrete sequence.
28      print(f"Minimum area = {required_area_mm2} mm^2")
```

Code 3.17: Discrete minimum-area search with a sentinel result.

The result is the smallest area in the tested sequence, not necessarily the exact mathematical minimum. It is also not a complete structural design. The step size of 50 mm^2 controls the search resolution.

3.17 A loop needs a defensible stopping rule

Examples of stopping rules include:

- all scheduled times have been processed;
- tank volume reaches capacity;
- temperature falls below a target;
- numerical error falls below a tolerance;
- the first acceptable design is found.

For every loop, ask:

1. Is termination guaranteed for all intended inputs?
2. Is the boundary inclusive or exclusive?
3. Could the selected step jump over the event?
4. Should the program impose a maximum number of passes?

Decision rule

A stopping condition is part of the model and specification. It should express why the repeated process is complete, not merely provide a convenient way to leave the loop.

3.18 Loop design checklist

Before coding:

1. What repeats?

2. Which values change?
3. What remains fixed?
4. Is a `for` or `while` loop clearer?
5. What is the stopping condition?

Before trusting the result:

1. trace a small case by hand;
2. test zero, one, and several passes;
3. inspect the last included value;
4. test failure to find a result;
5. interpret approximation, discretization, and step size.

3.19 Laboratory 1: branching and loops

The Week 2 slides recommend the following laboratory workflow:

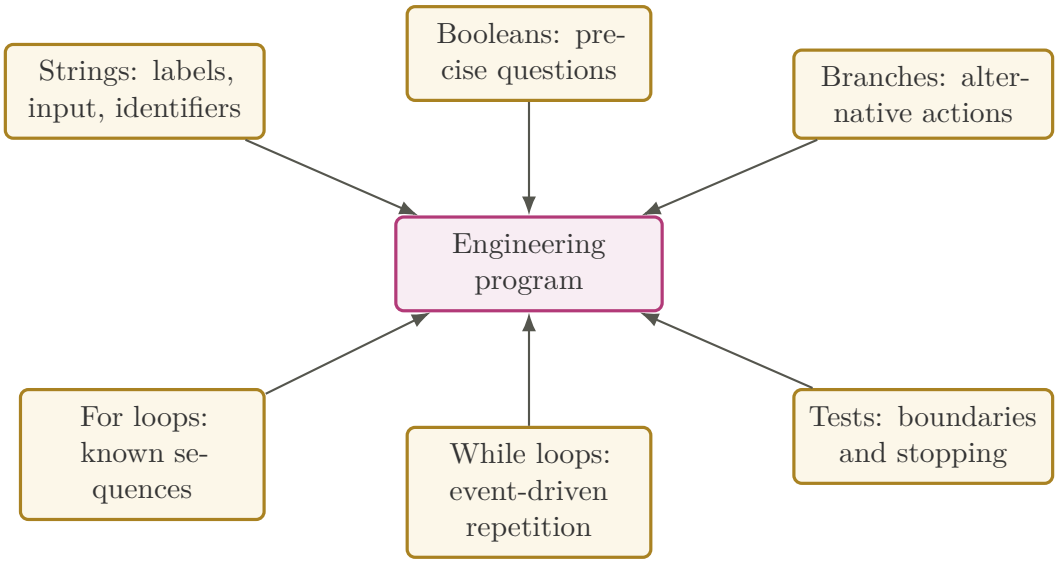
1. begin with a supplied or partially completed program;
2. identify inputs, units, decisions, and repeated steps;
3. predict outputs before running;
4. repair syntax, indentation, and logic errors;
5. modify one requirement;
6. test and explain the final result.

Possible contexts include a dropped object, tank filling, cooling, stress-based area search, equipment-ID validation, or operating-limit monitoring.

You should be able to trace, explain, and modify every submitted line. The authoritative Laboratory 1 instructions, collaboration rules, deadline, and submission format are those posted on Moodle.

3.20 Week 2 concept map

The combined capability developed across the three lectures is:



A Week 2 program can receive text, normalize or convert it, ask precise questions, choose a path, repeat calculations, and stop by a stated rule.

3.21 Week 2 exit ticket with solution

Consider:

```
# Initialize the running total before the loop.
total = 0

# Generate the integers 1, 2, 3, 4, and 5.
for i in range(1, 6):
    # Add the current loop value to the accumulator.
    total += i
```

Code 3.18: Exit-ticket accumulator loop.

The generated values are 1, 2, 3, 4, and 5. The loop has five passes. The final total is

$$1 + 2 + 3 + 4 + 5 = 15.$$

One useful boundary test is a sequence with one pass:

```
# Initialize the accumulator.
total = 0

# range(1, 2) generates only the value 1.
for i in range(1, 2):
    total += i

# The expected final total is 1.
print(total)
```

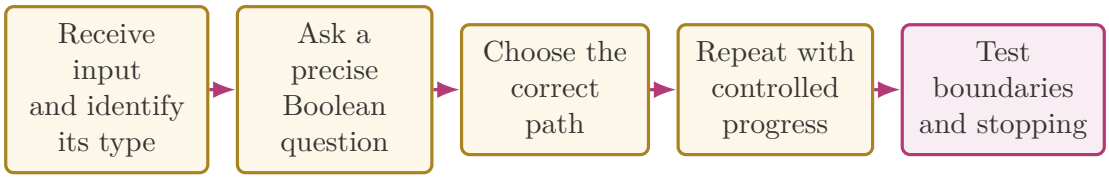
Code 3.19: Boundary test with exactly one loop pass.

A zero-pass test such as `range(1, 1)` is also useful; the total should remain at its initial value 0.

4 Week 2 Synthesis, Quick Reference, and Practice

4.1 The central Week 2 workflow

The complete Week 2 message can be summarized as:



A trustworthy control-flow program requires attention to both software behavior and engineering meaning:

- text input must be normalized or converted according to its intended meaning;
- every condition must correspond to a clear rule;
- comparison symbols must encode the intended boundaries;
- overlapping classification rules must be ordered correctly;
- invalid input should be rejected before risky calculations;
- every loop must have a clear repeated block and a defensible stopping rule;
- counters, accumulators, and running states must be initialized correctly;
- tests must cover every branch, important boundary, and loop failure case;
- a program classification remains conditional on the chosen model and rule.

4.2 Week 2 Python quick reference

Syntax	Meaning	Engineering note
`"text"`	Create a string literal.	Use for prompts, labels, units, identifiers, and messages.
`input(prompt)`	Read keyboard input as a string.	State the expected quantity, unit, or format in the prompt.
`float(text)`	Convert numerical text to a floating-point value.	Suitable for measured quantities when the text is valid.
`int(text)`	Convert numerical text to an integer.	Suitable for counts and whole-number indexes.

Syntax	Meaning	Engineering note
`strip()`	Remove surrounding whitespace.	Useful before comparing user-entered labels.
`lower()`	Convert letters to lowercase.	Use only when case normalization preserves meaning.
`len(s)`	Count characters in a string.	Includes spaces, punctuation, and hyphens.
`s[0]`	First character.	Python indexing begins at zero.
`s[-1]`	Last character.	Negative indexing counts from the end.
`==, !=`	Equality and inequality tests.	Do not confuse `==` with assignment `=`.
`<, <=, >, >=`	Ordered comparisons.	Choose inclusive or exclusive boundaries deliberately.
`and`	True only when both operands are true.	Use when every stated requirement must hold.
`or`	True when at least one operand is true.	Use for warnings triggered by any one of several conditions.
`not`	Reverse a Boolean value.	Useful for states such as `not sensor_connected`.
`if condition:`	Execute a block when the condition is true.	Indentation defines the block.
`elif condition:`	Test another condition after earlier ones fail.	Python selects the first true branch.
`else:`	Handle all cases not captured earlier.	No condition follows `else`.
`for x in range(...):`	Repeat over a known integer sequence.	The stop value is excluded.
`for character in text:`	Repeat over each character in a string.	The loop variable receives one character per pass.
`range(stop)`	Generate 0 through stop-1.	Useful for a known number of passes.
`range(start, stop)`	Generate start through stop-1.	The sequence includes start but excludes stop.
`range(start, stop, step)`	Generate a stepped sequence.	The sign of step must move toward the stopping side.
`while condition:`	Repeat while the condition is true.	Identify initialization, condition, and update.
`count += 1`	Increment a counter.	Initialize the counter before the loop.
`total += value`	Update an accumulator.	Do not reset the total inside the loop.
`state += rate * dt`	Advance a running state.	Check units and the effect of step size.
`break`	Exit the nearest loop.	Useful after the search goal is found.

Syntax	Meaning	Engineering note
`continue`	Skip the rest of the current pass.	Useful for clearly identified invalid or irrelevant cases.
`None`	Represent absence of a useful result.	Can indicate that a search has not yet succeeded.
`abs(a-b) < tol`	Tolerance-based comparison.	Select tolerance according to scale and required accuracy.

4.3 Decision-program checklist

Before coding a decision:

1. State the rule in plain language.
2. Identify each individual yes-or-no question.
3. Decide whether boundaries are inclusive or exclusive.
4. Identify invalid inputs that must be rejected first.
5. Order overlapping categories from restrictive to broad.
6. Decide which numerical evidence should appear in the output.

Before trusting a decision:

1. Test a representative value in every branch.
2. Test exactly at each boundary.
3. Test just below and just above every important threshold.
4. Test invalid input before the main calculation.
5. Test an extreme value that reaches the final `else` branch.
6. Explain what the classification establishes and what it does not establish.

4.4 Loop-program checklist

Before coding a loop:

1. What repeats?
2. What changes on each pass?
3. What remains fixed?
4. Is the sequence known, suggesting `for`?
5. Is repetition event-driven, suggesting `while`?
6. What value or event ends the process?

Before trusting a loop:

1. Trace a small case by hand.
2. Test zero passes, one pass, and several passes.
3. Check the last included value of `range()`.
4. Confirm that an update moves a `while` loop toward termination.
5. Test a search in which no candidate succeeds.
6. Examine whether the step size can jump over the event.
7. Consider whether a maximum-pass limit is needed.

4.5 Glossary

Accumulator
A variable that stores a running total across loop passes.

Boolean
A value of type `bool`: either `True` or `False`.

Boundary case
An input exactly at an important threshold or endpoint.

Branch
One possible path through a conditional structure.

Chained comparison
A compact interval expression such as `20 <= T <= 80`.

Condition
A Boolean expression that controls a branch or loop.

Conditional
A program structure that chooses actions based on a Boolean result.

Counter
A variable that counts events or cases, often updated with `+= 1`.

Iteration
Repeated execution of a block of statements.

Loop body
The indented statements repeated by a loop.

Loop variable
A name successively associated with values from an iterable.

Normalization
Converting equivalent input forms to a consistent representation.

Range
A Python object that generates a progression of integers for iteration.

Running state
A system quantity updated from one loop pass to the next.

Sentinel result
A special value such as `None` used to represent "not found yet."

Stopping condition
The rule that determines when repetition ends.

String
An ordered sequence of characters represented by type `str`.

Tolerance
An allowed numerical difference used when exact floating-point equality is inappropriate.

Truthiness
Python's interpretation of certain non-Boolean values as true or false in a condition.

Validation
Checking that input satisfies required rules before using it.

While loop
A loop that repeats while a Boolean condition remains true.

4.6 Cumulative practice

Attempt the questions before reading the solutions. Predict, trace, and explain rather than relying only on execution.

4.6.1 Concept questions

1. Why does `input()` return a string even when the user types digits?
2. Explain the difference between `=` and `==`.
3. Why can `pressure < limit` and `pressure <= limit` produce different engineering decisions?
4. Why should input validation occur before division?
5. Why can incorrect `elif` ordering make a program misclassify a value?
6. Explain the difference between `and` and `or` using an engineering operating-limit example.
7. Why can exact floating-point equality be unreliable?
8. Why is the stop argument of `range()` useful even though it is excluded?
9. What distinguishes a counter from an accumulator?
10. What three elements should be visible in a well-designed `while` loop?
11. Why might a smaller simulation step improve accuracy but increase computational work?
12. What does a discrete design-search result establish, and what does it not establish?

4.6.2 Code tracing: strings and Boolean values

```
# Store digits as text.
value_text = "8"
```

```
3
4  # Repeat the text twice.
5  print(value_text * 2)
6
7  # Convert the text to an integer and multiply numerically.
8  print(int(value_text) * 2)
9
10 # Compare the string with another string.
11 print(value_text == "8")
12
13 # Compare the converted integer with a numerical boundary.
14 print(int(value_text) >= 10)
```

Code 4.1: Practice A: strings, conversion, and comparison.

4.6.3 Code tracing: branch order

```
1  # Store a measured pressure in kPa.
2  pressure_kpa = 100.0
3
4  # Classify using ordered thresholds.
5  if pressure_kpa < 100.0:
6      category = "low"
7  elif pressure_kpa <= 200.0:
8      category = "normal"
9  else:
10     category = "high"
11
12 # Display the selected category.
13 print(category)
```

Code 4.2: Practice B: determine which category is selected.

4.6.4 Code tracing: range and accumulator

```
1  # Initialize a running total.
2  total = 0
3
4  # Generate 2, 4, 6, and 8.
5  for value in range(2, 10, 2):
6      # Add the current value to the accumulator.
7      total += value
8
9      # Display the current value and running total.
10     print(value, total)
```

Code 4.3: Practice C: trace every pass and final total.

4.6.5 Debugging practice: Boolean logic

```
1  # Store a material name entered by a user.
2  material = "aluminum"
3
```

```
# LOGIC ERROR: only the first comparison is explicit.
if material == "steel" or "aluminum":
    print("Recognized material")
else:
    print("Unknown material")
```

Code 4.4: Practice D: faulty material-recognition program.

Repair the condition and explain why the original is wrong.

4.6.6 Debugging practice: loop progress

```
# Initialize temperature above the target in deg C.
temperature_c = 100.0

# Repeat while the temperature remains above 40 deg C.
while temperature_c > 40.0:
    # Display the current state.
    print(temperature_c)

    # BUG: the update is missing, so the state never approaches 40 deg C.
```

Code 4.5: Practice E: faulty cooling loop.

Add a simple update that decreases the temperature by 5 °C per pass. Then identify the final printed temperature and the value at which the loop stops.

4.6.7 Programming practice: operating-state classifier

Write a program that classifies a battery voltage as:

- below 10 V: low;
- 10 V to 12.6 V inclusive: normal;
- above 12.6 V: high.

The program should read input, validate that voltage is nonnegative, report the numerical value, and print the category.

4.6.8 Programming practice: repeated energy calculation

A device operates at 2 kW. Print cumulative energy after each hour from hour 1 through hour 6. Use a `for` loop and an accumulator. Predict the final total before running.

4.6.9 Programming practice: event-driven tank filling

A tank begins with 20 L, fills at 5 L/min, and has a target of 60 L. Use a `while` loop with a one-minute step. Print time and volume after every update. Explain why the loop terminates.

4.6.10 Engineering interpretation

A design search reports `Minimum area = 350 mm^2`. The search tested areas from 100 to 1000 mm^2 in steps of 50 mm^2. Write three sentences explaining what the result means and at least three limitations.

4.7 Practice solutions

4.7.1 Concept responses

1. Keyboard input arrives as characters, and Python cannot infer whether digits represent a number, identifier, date, or label.
2. `=` assigns a name to an object; `==` asks whether two values are equal.
3. The first rule excludes equality, while the second includes it; the selected boundary determines the category at exactly the limit.
4. Validation prevents meaningless operations such as division by zero and produces a clearer explanation for the user.
5. Python chooses the first true branch. A broad condition placed too early can capture values intended for a later, more restrictive category.
6. `and` requires every stated condition to be true; `or` requires at least one. For example, a system may be normal only when temperature and pressure are both within limits, but a warning may occur when temperature or pressure exceeds its limit.
7. Many decimal values are stored as binary approximations, so mathematically equal quantities may not have identical stored values.
8. Excluding the stop value makes interval lengths and adjacent ranges predictable; for example, `range(0, 11)` naturally includes 0 through 10.
9. A counter records how many events occurred; an accumulator stores a running total of values.
10. Initialization, condition, and update.
11. A smaller step samples the process more finely but requires more loop passes.
12. It identifies the first passing candidate in the tested discrete sequence. It does not prove an exact continuous optimum or a complete safe design.

4.7.2 Tracing solutions

Practice A outputs `88`, `16`, `True`, and `False`.

Practice B prints `normal`. Equality at 100 kPa fails the first strict test but passes the second inclusive test.

Practice C traces as follows:

Pass	Current value	Running total
1	2	2
2	4	6
3	6	12
4	8	20

4.7.3 Boolean-logic repair

```
# Store the material label to be classified.
material = "aluminum"

# Compare the material explicitly with each recognized option.
if material == "steel" or material == "aluminum":
    print("Recognized material")
else:
    print("Unknown material")
```

Code 4.6: Corrected material-recognition program.

The original condition was wrong because the nonempty string `"aluminum"` was used as a truthy value rather than as the right operand of a comparison.

4.7.4 Cooling-loop repair

```
# Initialize temperature in deg C.
temperature_c = 100.0

# Repeat while temperature remains above the 40 deg C target.
while temperature_c > 40.0:
    # Display the current state before updating it.
    print(temperature_c)

    # Decrease temperature by 5 deg C per pass.
    temperature_c -= 5.0

# Report the first value that makes the loop condition False.
print(f"Loop stopped at {temperature_c:.1f} deg C")
```

Code 4.7: Corrected cooling loop with a constant decrement.

The last temperature printed inside the loop is 45 °C. The update then produces 40 °C, making `temperature_c > 40.0` false.

4.7.5 Operating-state classifier solution

```
# Read battery voltage in volts and convert text to float.
voltage_v = float(input("Enter battery voltage in V: "))

# Validate the physical domain used in this exercise.
if voltage_v < 0:
    print("ERROR: voltage must not be negative")
else:
    # Classify using ordered and explicit boundaries.
```

```
    if voltage_v < 10.0:
        category = "low"
    elif voltage_v <= 12.6:
        category = "normal"
    else:
        category = "high"

    # Report both evidence and classification.
    print(f"Voltage = {voltage_v:.2f} V")
    print(f"Category = {category}")
```

Code 4.8: Annotated battery-voltage classifier.

4.7.6 Repeated energy solution

```
# Store constant power in kW.
power_kw = 2.0

# Store the interval duration in hours.
interval_h = 1.0

# Initialize cumulative energy in kWh.
total_energy_kwh = 0.0

# Process hours 1 through 6.
for hour in range(1, 7):
    # Add energy used during the current interval.
    total_energy_kwh += power_kw * interval_h

    # Report the cumulative energy after this hour.
    print(f"Hour {hour}: {total_energy_kwh:.1f} kWh")
```

Code 4.9: Cumulative energy after six one-hour intervals.

The final total is 12 kW h.

4.7.7 Tank-filling solution

```
# Store the target volume in litres.
target_volume_l = 60.0

# Store the constant filling rate in litres per minute.
flow_rate_lpm = 5.0

# Initialize the tank with 20 litres.
volume_l = 20.0

# Initialize elapsed time in minutes.
time_min = 0

# Repeat until the target volume is reached.
while volume_l < target_volume_l:
    # Advance time by one minute.
    time_min += 1

```

```
18      # Add one minute of inflow to the running volume.
19      volume_l += flow_rate_lpm
20
21      # Report the updated state.
22      print(f"Time = {time_min} min, volume = {volume_l:.1f} L")
```

Code 4.10: Event-driven tank-filling loop.

The loop terminates because `volume_l` increases by 5 L on every pass and eventually reaches the 60 L target.

4.7.8 Engineering interpretation response

A suitable response is:

> The program found that 350 mm^2 was the first candidate in the tested sequence that satisfied the stated numerical rule. Because candidates were separated by 50 mm^2, the result is a discrete search outcome rather than an exact continuous minimum. It does not by itself establish complete structural safety, manufacturing feasibility, optimal cost, or adequacy under unmodeled loading and failure modes.

Source Basis and Suggested Reading

These notes were developed to accompany the complete Week 2 lecture-slide set and preserve its organization, terminology, examples, engineering cautions, activities, and level of detail. The principal sources are:

1. Yanwei Wang, *Programming for Engineers - Week 2: Strings, Branching, Conditionals, Indentation, and Iteration*, complete lecture-slide set, School of Engineering, Nazarbayev University, Fall 2026.
2. Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 2.
3. Yanwei Wang, *Programming for Engineers with Python*, working course-manuscript draft, Chapters 2–4 and associated instructor notes.
4. John V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Sections 2.2–2.4.

Primary preparation for students

For every short example:

1. translate each condition into a question in plain language;
2. predict the branch or generated sequence before execution;
3. trace changed values by hand;
4. run the program;
5. modify one boundary, input, or stopping condition;
6. test representative, boundary, and invalid cases;
7. explain what the program concludes and what remains outside its model.

Key idea

Programs become useful when they can decide and repeat. Make the rule explicit. Make the boundary testable. Make the loop progress. Keep the engineer accountable.